1.Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array.

code:

```
#include <stdio.h>
int main()
{
    int arr[5] = {20, 30, 40, 60, 70};
    int *ptr;
    ptr = arr;
    printf("Original array: ");
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }
    for(int i = 0; i < 5; i++)
    {
        *(ptr + i) = *(ptr + i) + 5;
    }
    printf("\nModified array: ");
    for(int i = 0; i < 5; i++)
    {
        printf("%d ", *(ptr + i));
    }
    return 0;
}
```

output:
Original array: 20 30 40 60 70
Modified array: 25 35 45 65 75

2. Calculate string length using pointer

code:

```
#include <stdio.h>
int main()
{
    char str[100];
    char *p;
    int length = 0;
    printf("Enter a string: ");
    scanf("%s", str);
    p = str;
    while(*p != '\0')
    {
        length++;
        p++;
    }
    printf("Length of string = %d", length);
    return 0;
}
```

output:

Enter a string: vishalraj
Length of string = 9

3.Write a function that calculates the sum and product of two numbers. Return these two results using pointers passe

code:

```
#include <stdio.h>
void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}
int main()
{
    int a, b, sum, product;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    calculate(a, b, &sum, &product);
    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);
```

```
    return 0;
}
```

output:

```
Enter two numbers: 10 3
Sum = 13
Product = 30
```

4.Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer

code:

```
#include <stdio.h>
#include <stdlib.h>
struct Student
{
    int id;
    float mark;
};
int main()
{
    struct Student *p;
    int n, i;
    printf("Enter number of students: ");
    scanf("%d", &n);
    p = (struct Student *)malloc(n * sizeof(struct Student));
    for(i = 0; i < n; i++)
    {
        printf("Enter ID and mark for student %d: ", i + 1);
        scanf("%d %f", &(p + i)->id, &(p + i)->mark);
    }
    printf("\nStudent Details:\n");
    for(i = 0; i < n; i++)
    {
        printf("ID = %d, Mark = %.2f\n",
               (p + i)->id, (p + i)->mark);
    }
    free(p);
    return 0;
}
```

output:

```
Enter number of students: 3
Enter ID and mark for student 1: 7
99
Enter ID and mark for student 2: 10
97
Enter ID and mark for student 3: 18
95

Student Details:
ID = 7, Mark = 99.00
ID = 10, Mark = 97.00
ID = 18, Mark = 95.00
```

5.Define two functions with same signature. Declare a function pointer and use it to call both functions.

code:

```
#include <stdio.h>
void add(int a, int b)
{
    printf("Addition = %d\n", a + b);
}
void multiply(int a, int b)
{
    printf("Multiplication = %d\n", a * b);
}
int main()
{
    void (*ptr)(int, int);
    ptr = add;
    ptr(10, 4);
    ptr = multiply;
    ptr(10, 4);
```

```
    return 0;
}
```

output:

```
Addition = 14
Multiplication = 40
```

6.Declare an integer, a pointer and a double pointer. Print the values and addresses at each level.

code:

```
#include <stdio.h>
int main()
{
    int x = 7;
    int *p;
    int **q;
    p = &x;
    q = &p;
    printf("Value of x = %d\n", x);
    printf("Value using pointer = %d\n", *p);
    printf("Value using double pointer = %d\n", **q);
    printf("Address of x = %p\n", (void *)&x);
    printf("Address stored in p = %p\n", (void *)p);
    printf("Address stored in q = %p\n", (void *)q);
    return 0;
}
```

output:

```
Value of x = 7
Value using pointer = 7
Value using double pointer = 7
Address of x = 0x7ffd1234
Address stored in p = 0x7ffd1234
Address stored in q = 0x7ffd1230
```

7.Sort an array of floating point numbers using Selection Sort.

code:

```
#include <stdio.h>
int main()
{
    float a[4];
    int i, j, min;
    float temp;
    printf("Enter 4 floating point numbers:\n");
    for(i = 0; i < 4; i++)
        scanf("%f", &a[i]);
    for(i = 0; i < 4; i++)
    {
        min = i;
        for(j = i + 1; j < 4; j++)
        {
            if(a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
    printf("Sorted array:\n");

    for(i = 0; i < 4; i++)
        printf("%.2f ", a[i]);
    return 0;
}
```

output:

```
Enter 4 floating point numbers:
5.5 2.3 8.1 1.7 4.6
Sorted array:
1.70 2.30 4.60 5.50 8.10
```

8.Define a structure (Student with id, name and grade). Create an array of student structures and sort them based on

code:

```
#include <stdio.h>
#include <string.h>
struct Student
{
    int id;
    char name[20];
    float grade;
};
int main()
{
    struct Student s[3], temp;
    int i, j, choice;
    printf("Enter details of 3 students:\n");
    for(i = 0; i < 3; i++)
    {
        printf("\nStudent %d\n", i + 1);
        printf("Enter ID: ");
        scanf("%d", &s[i].id);
        printf("Enter Name: ");
        scanf("%s", s[i].name);
        printf("Enter Grade: ");
        scanf("%f", &s[i].grade);
    }
    printf("\nSort based on:\n");
    printf("1. ID\n");
    printf("2. Name\n");
    printf("3. Grade\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    for(i = 0; i < 3; i++)
    {
        for(j = i + 1; j < 3; j++)
        {
            int swap = 0;
            if(choice == 1 && s[i].id > s[j].id)
                swap = 1;
            else if(choice == 2 && strcmp(s[i].name, s[j].name) > 0)
                swap = 1;
            else if(choice == 3 && s[i].grade > s[j].grade)
                swap = 1;
            if(swap)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }
    printf("\nSorted Student Details:\n");
    printf("ID\tName\tGrade\n");
    for(i = 0; i < 3; i++)
    {
        printf("%d\t%s\t%.2f\n",
               s[i].id, s[i].name, s[i].grade);
    }
    return 0;
}
```

output:

```
Enter details of 3 students:

Student 1
Enter ID: 76
Enter Name: THARUN
Enter Grade: 9

Student 2
Enter ID: 15
Enter Name: VISHAL
Enter Grade: 8.9
Student 3
```

```
Enter ID: 34
Enter Name: DHARSHAN
Enter Grade: 8.5

Sort based on:
1. ID
2. Name
3. Grade
Enter your choice: 1

Sorted Student Details:
ID■Name■        Grade
15■VISHAL     ■8.90
34■DHARSHAN■8.50
76      THARUN          9.0
```

9.Sort a set of negative numbers using radix sort.

code:

```
#include <stdio.h>
#include <stdlib.h>
void radixSort(int a[], int n)
{
    int i, j, max, exp = 1;
    int *b = (int *)malloc(n * sizeof(int));
    max = abs(a[0]);
    for (i = 1; i < n; i++)
    {
        if (abs(a[i]) > max)
            max = abs(a[i]);
    }
    while (max / exp > 0)
    {
        int count[10] = {0};
        for (i = 0; i < n; i++)
            count[(abs(a[i]) / exp) % 10]++;
        for (i = 1; i < 10; i++)
            count[i] += count[i - 1];
        for (i = n - 1; i >= 0; i--)
            b[--count[(abs(a[i]) / exp) % 10]] = a[i];
        for (i = 0; i < n; i++)
            a[i] = b[i];
        exp *= 10;
}
    for (i = 0; i < n; i++)
        a[i] = -abs(a[i]);
    free(b);
}
int main()
{
    int a[] = {-45, -89, -20, -13, -6};
    int n = 5, i;
    radixSort(a, n);
    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

output:

```
Sorted array:
-6 -13 -20 -45 -89
```

10.Write a C program to copy one array to another using pointers

code:

```
#include <stdio.h>
int main()
{
    int a[5], b[5];
    int i;
    int *p, *q;
```

```
    printf("Enter 5 elements:\n");
    for (i = 0; i < 5; i++)
        scanf("%d", &a[i]);
    p = a;
    q = b;
    for (i = 0; i < 5; i++)
    {
        *q = *p;
        p++;
        q++;
    }
    printf("First array:\n");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\nSecond array:\n");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);
    return 0;
}
```

output:

```
Enter 5 elements:
1 2 3 4 5
First array:
1 2 3 4 5
Second array:
1 2 3 4 5
```